Coordinate-wise SVRG for Low Signal-to-Noise Objectives: Theory, Tuned Ablations, and Negative Results

Kunal

Technical report (draft)

Abstract

I implement a variance-reduced optimizer with adaptive coordinate scaling ("Coordinate SVRG") in PyTorch and JAX, validate both against an independent NumPy oracle, and evaluate it on synthetic low signal-to-noise finite-sum problems built to resemble noisy tick data, plus three small real datasets. I prove a variance bound that explains the method's behaviour, a staleness-controlled variant of it for the adaptive engine, a numerical-safety property, and a short corollary explaining when diagonal scaling helps. Against learning-rate-tuned Adam and SGD with momentum (constant and cosine-decay), variance reduction reaches float64 round-off on well-scaled convex problems; adaptive scaling helps only when features are badly scaled; and on a noisy non-convex fit the more precisely optimised model generalises *worse*. The preconditioning corollary also yields a simpler method, Jacobi-preconditioned SVRG, whose hyper-parameters can be read off the theory with no tuning; on badly scaled least squares it reaches round-off on every seed, where the best tuned adaptive variant reaches 3.5×10^{-11}. Ablating the invariants shows that misaligned batches are worse than no variance reduction at all and that textbook Adam fails on adversarial inputs where the full guard set never does. On three real datasets with raw, unstandardised features, preconditioning still dominates, but the round-off result does not carry and the margin over tuned baselines shrinks to between 0.06 and 0.52 decades; after standardising the same features the round-off result reproduces, so the shortfall was raw-scale conditioning, not real-data difficulty. With validation-tuned learning rates the held-out deficit of the adaptive engine tracks adaptive scaling (Adam shows it too), not variance reduction. There is no wall-clock win over a direct solve at the sizes tested, and no benefit for walk-forward signal tracking. Apart from three small real datasets and one walk-forward study of ten ETFs the data are synthetic, all models are small, and the PyTorch and JAX engines did not produce the experiment numbers; the report states these limits explicitly. The ETF study finds no tradable signal and no effect of the optimizer; on a T4 GPU, SVRG ties the best direct solve only at the largest size tested.

1 Introduction

Stochastic gradient methods stall on noisy finite-sum objectives because per-sample gradients do not vanish at the optimum, so mini-batch variance is bounded below. Variance-reduced methods such as SVRG [1] remove that floor with a control variate built from a periodic snapshot. Quantitative-finance estimation problems (calibrating linear and generalised-linear signal models on noisy tick data) are finite sums with very low signal-to-noise, which makes the question natural: does variance reduction, combined with Adam-style coordinate scaling [2], help there?

Contributions.

1. A reference implementation in PyTorch and JAX with four enforced invariants (Section 2), tested step-for-step against an independent NumPy oracle, with adversarial-input fuzz tests.
2. Four short results (Section 3): a variance bound (Lemma 2), a staleness-controlled version valid for the adaptive engine (Proposition 3), a numerical-safety property (Proposition 4), and a preconditioning corollary (Proposition 5) that predicts when adaptive scaling helps.
3. A learning-rate-tuned ablation with held-out seeds and paired bootstrap intervals across fourteen experiments, including negative and null results.
4. The frozen-preconditioner mode is implemented in both engines (`set_frozen_preconditioner` in PyTorch, a `preconditioner` argument in JAX), with differential tests against the oracle.
5. A falsifiable prediction from the theory, tested: a method with theory-prescribed hyper-parameters (Section 5.6) that outperforms the more elaborate adaptive engine on badly scaled problems. The engine's own scaling is best read as a gradient-only proxy for it.

I make no claim of a tradable edge in real markets, of results for deep networks, or of suitability for latency-critical production use.

2 Method

Let $F(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(w)$. Given a snapshot $\tilde{w}$ with full gradient $\mu = \nabla F(\tilde{w})$ and a mini-batch B, the variance-reduced gradient is

$$\hat{g}(w) = g_B(w) - g_B(\tilde{w}) + \mu, \qquad g_B = \tfrac{1}{|B|}\textstyle\sum_{i\in B} \nabla f_i.$$

The engine then applies coordinate-wise momentum and second-moment tracking,

$$m \leftarrow \beta_1 m + (1-\beta_1)\hat{g}, \quad v \leftarrow \beta_2 v + (1-\beta_2)\hat{g}^2, \quad w \leftarrow w - \eta\,\mathrm{clip}\Big(\frac{m/(1-\beta_1^t)}{\max\{\sqrt{v/(1-\beta_2^t)},\,\varepsilon_{\text{floor}}\}}, \pm c\Big).$$

Four invariants are enforced and tested: (1) the variance of $\hat{g}$ vanishes as $w \to \tilde{w}$; (2) $g_B(w)$ and $g_B(\tilde{w})$ use the *same* batch (one batch identifier is passed to both evaluations); (3) in PyTorch, parameters, snapshot, μ, m, v live in contiguous flat buffers; (4) gradients are sanitised and bounded, the denominator is floored and the ratio clipped, so no division can produce NaN or Inf; Section 6 shows that the gradient bound is the operative guard.

3 Theory

Let each f_i have L_i-Lipschitz gradient and write $\bar{L}^2 = \frac{1}{n}\sum_i L_i^2$.

Lemma 1 (Unbiasedness)**.** *For any snapshot,* $\mathbb{E}_B[\hat{g}(w)] = \nabla F(w)$.

Proof. $\mathbb{E}_B g_B(w) = \nabla F(w)$ and $\mathbb{E}_B g_B(\tilde{w}) = \mu$. □

Lemma 2 (Variance bound)**.** *For uniform mini-batches of size b (with or without replacement),*

$$\mathbb{E}\|\hat{g}(w) - \nabla F(w)\|^2 \;\le\; \frac{\bar{L}^2}{b}\,\|w - \tilde{w}\|^2.$$

For least squares the error is exactly $(H_B - H)(w - \tilde{w})$, so scaling the displacement by s scales the variance by exactly s^2.

Proof. With $Y_i = \nabla f_i(w) - \nabla f_i(\tilde{w})$, $\hat{g} - \nabla F = \bar{Y}_B - \bar{Y}$, a sample-mean error. Its variance is σ^2/b (times $\frac{n-b}{n-1} \le 1$ without replacement) with $\sigma^2 = \frac{1}{n}\sum_i \|Y_i - \bar{Y}\|^2 \le \frac{1}{n}\sum_i \|Y_i\|^2 \le \bar{L}^2\|w - \tilde{w}\|^2$. □

Plain mini-batch SGD has no such control: at a stationary point of a noisy problem, per-sample gradients do not vanish, so its variance has a positive floor. The variance of $\hat{g}$ depends on the snapshot distance, not on the data noise; this is why the snapshot must be refreshed.

Proposition 3 (Staleness-controlled variance)**.** *If the normalised update is clipped to $\pm c$ per coordinate, then K steps after a snapshot,*

$$\mathbb{E}\|\hat{g} - \nabla F\|^2 \le \frac{\bar{L}^2\, d}{b}\,(\eta c K)^2.$$

Proof. Each coordinate moves at most ηc per step, so $\|w - \tilde{w}\|_2^2 \le d(\eta c K)^2$; apply Lemma 2. □

This holds for any adaptive normalisation provided the step is clipped. It is very loose: across snapshot intervals from 8 to 1024, measured variance was at most 2.2×10^{-4} of the bound.

Proposition 4 (Numerical safety)**.** *Let $G = \frac{1}{4}\sqrt{realmax}$ for the working precision. After sanitising NaN$\to 0$ and $\pm\infty \to \pm G$ and clipping to $[-G, G]$, $|m_t| \le G$ and $0 \le v_t \le G^2$ by induction; the denominator is at least $\varepsilon_{\text{floor}} > 0$ and the final ratio is clipped, so every step is finite with $|\Delta w_j| \le \eta c$. In single precision the unclipped quotient is itself finite whenever $1 - \beta_1 \ge 10^{-4}$.*

This is checked by randomised adversarial tests (zeros, denormals, $10^{\pm 38}$, $\pm\infty$, NaN) in float32 and float64.

Proposition 5 (Frozen diagonal preconditioner)**.** *Let $D \succ 0$ be diagonal and fixed, and consider $w \leftarrow w - \eta D^{-1}\hat{g}$. If each f_i is convex, $\tilde{f}_i(u) = f_i(D^{-1/2}u)$ are L_D-smooth and $\tilde{F}$ is γ_D-strongly convex, the Johnson–Zhang theorem [1] holds verbatim with (L_D, γ_D): for step $\eta < 1/(2L_D)$ and m inner steps, $\mathbb{E}[F(\tilde{w}_s) - F(w^*)] \le \alpha_D^s\,(F(\tilde{w}_0) - F(w^*))$ with*

$$\alpha_D = \frac{1}{\gamma_D \eta (1 - 2L_D\eta) m} + \frac{2L_D\eta}{1 - 2L_D\eta}.$$

Proof. Put $u = D^{1/2}w$. Then $\nabla \tilde{f}_i(u) = D^{-1/2}\nabla f_i(w)$, so the SVRG step in u is $D^{1/2}$ applied to the preconditioned step in w: the iterations coincide up to a linear change of coordinates (verified numerically in the test suite), with equal function values. □

The rate depends on $\kappa_D = L_D/\gamma_D$. For least squares with Jacobi scaling $D = \operatorname{diag}(X^\top X/n)$, κ_D is invariant to rescaling feature columns, and Jacobi scaling brings the condition number of the preconditioned *Hessian* within a factor of the dimension of the best diagonal scaling [5] (a statement about the Hessian, not about κ_D's per-sample constant). On the badly scaled problem below, κ falls from 1.1×10^7 to 82; on equal-variance features it does not move ($81 \to 82$).

Remark. The theorem is for Option II snapshots (a uniformly random inner iterate); the engine uses the last iterate. The engine's preconditioner $\operatorname{diag}(\sqrt{\hat{v}_t})$ is not frozen and tracks gradient scale rather than Hessian-diagonal scale. The proposition covers the frozen idealisation; the link to the adaptive engine is empirical consistency, not a theorem. **I prove no convergence theorem for the adaptive engine itself.** Adam-type methods can fail to converge in general [3], and normalisation keeps the step near η per coordinate even as gradients shrink, which explains the constant-rate plateau seen in Section 5.

4 Experimental protocol

All results use a NumPy reference implementation of the update rule; the PyTorch and JAX engines are tested against it (step-for-step differential tests, run on CPU and on a GPU runtime) but *did not produce the numbers reported here.* Except in Section 5.7, data come from a simulator: AR(1) features, a hidden alpha with drift and regime switches, Gaussian noise at three levels, heavy-tailed toxic-flow shocks and arbitrage jumps (empirical SNR about -4 dB in the default regime). Every method receives its own learning-rate sweep on tuning seeds and is scored on disjoint held-out seeds. Budgets count sample-gradient evaluations; SVRG is charged for double gradients and every full-gradient snapshot pass. Baselines are Adam (with the same clamped denominator) and SGD with momentum, each with constant and cosine-decay learning rates. Differences are reported in decades of $\log_{10}$(final gap) with 95% paired bootstrap intervals over seeds and win counts; no multiple-comparison correction is applied. Where one method sits at float64 round-off ($\approx 10^{-31}$), the *magnitude* of a difference is not meaningful, only the win count.

5 Results

5.1 Variance reduction dominates on well-scaled convex problems

On least squares (volatile regime, Table 1) Coordinate SVRG reaches a median gap of 4.1×10^{-31}, at float64 round-off, and beats the best tuned baseline (SGD + momentum, cosine lr, 9.7×10^{-8}) on 20/20 held-out seeds. Across the three noise regimes the best baseline stops at 8.3×10^{-8} to 2.8×10^{-7}. On ℓ_2-regularised logistic regression for the sign of the return the win count is 12/12.

Method	tuned lr	median final gap	evals to target
SGD + momentum	1.8×10^{-3}	7.8×10^{-6}	not reached
SGD + momentum, cosine lr	4.2×10^{-3}	9.7×10^{-8}	not reached
Adam (clamped)	3.2×10^{-4}	4.2×10^{-6}	not reached
Adam (clamped), cosine lr	7.5×10^{-4}	9.7×10^{-8}	not reached
SVRG + momentum	1.3×10^{-1}	3.9×10^{-31}	72 448
Coordinate SVRG (ours)	1.8×10^{-3}	4.1×10^{-31}	85 376
Coordinate SVRG, cosine lr (ours)	1.0×10^{-2}	4.0×10^{-31}	85 376

Table 1: Least squares, volatile regime. Median final loss gap and sample-gradient evaluations to reach 10^{-8} of the initial gap, held-out seeds. †: best learning rate on the edge of the grid.

Method	tuned lr	median final gap	evals to target
SGD + momentum	3.2×10^{-4}	5.3×10^{-2}	not reached
SGD + momentum, cosine lr	1.0×10^{-2}	1.7×10^{-2}	not reached
Adam (clamped)	7.5×10^{-4}	4.3×10^{-3}	not reached
Adam (clamped), cosine lr	1.0×10^{-2}	5.9×10^{-6}	not reached
SVRG + momentum	1.0×10^{-2}	2.2×10^{-2}	not reached
Coordinate SVRG (ours)	1.0×10^{-2}	6.8×10^{-4}	not reached
Coordinate SVRG, cosine lr (ours)	5.6×10^{-2}	3.5×10^{-11}	348 288

Table 2: Least squares with feature scales spread over three decades. Same protocol as Table 1.

Problem	Method	median final gap	evals to target
well-scaled	Jacobi SVRG, theory-prescribed	6.2×10^{-30}	75 192
well-scaled	Jacobi SVRG, tuned lr	4.0×10^{-31}	49 152
badly scaled	Jacobi SVRG, theory-prescribed	2.6×10^{-29}	73 848
badly scaled	Jacobi SVRG, tuned lr	1.8×10^{-29}	49 152

Table 3: Jacobi-preconditioned SVRG on the datasets and seeds of Tables 1 and 2. The theory-prescribed row uses no tuning at all.

Dataset	decades	cond(H) raw $\to$ Jacobi	best baseline	Coordinate SVRG (cos)	Jacobi SVRG
breast cancer	5.3	$3.1 \times 10^{7} \to 2.5 \times 10^{5}$	6.9×10^{-3}	6.0×10^{-3}	6.0×10^{-3}
wine	3.4	$1.1 \times 10^{7} \to 4.0 \times 10^{3}$	5.7×10^{-3}	8.5×10^{-3}	1.7×10^{-3}
diabetes	1.8	$5.2 \times 10^{7} \to 4.0 \times 10^{4}$	2.4×10^{1}	4.9×10^{1}	1.8×10^{1}

Table 4: Real datasets with raw, unstandardised features (plus an intercept): median final loss gap after 1000 epochs of sample-gradient evaluations, learning rates tuned per method. No method reached the 10^{-8} target on any dataset.

5.2 Adaptive scaling helps only when features are badly scaled

With column scales spread over three decades (Table 2), Coordinate SVRG at a constant rate has a median paired difference of -1.5 decades against SVRG with plain momentum (negative favours Coordinate SVRG; 95% CI [-2.0, -1.1], better on 20/20 seeds), yet plateaus at a median gap of 6.8×10^{-4}, as anticipated in Section 3. Adding cosine decay removes the plateau (median 3.5×10^{-11}) and beats the best tuned baseline, Adam with cosine decay (5.9×10^{-6}): paired difference -5.3 decades (CI [-6.6, -4.9], better on 20/20 seeds). On well-scaled data the adaptive component does not help and, at constant rate on the larger problem, hurts. Proposition 5 predicts this pattern through κ_D.

5.3 The variance law along a real trajectory

Along one run (Figure 3), mini-batch SGD variance stays between 1.7 and 2.5, while the variance of $\hat{g}$ falls from 2.6×10^{-1} to 9.9×10^{-16}. In all 25 probes, each at worst-case snapshot staleness, measured variance was within the Lemma 2 bound (100%) and the Proposition 3 bound (100%).

5.4 Non-convex fit: sharper stationarity, worse generalisation

For a one-hidden-layer tanh network the squared full-gradient norm of Coordinate SVRG with cosine decay is lower than the best baseline's by -1.3 decades in $\log_{10}$ (negative means lower; 8/8 seeds). On a held-out random split, however, with learning rates tuned on training loss, its test loss is 0.1445 against 0.1317 for SGD + momentum, cosine lr (paired difference +0.0135, 95% CI [+0.0106, +0.0150]; the noise floor is 0.125), while its training loss is lower (0.1079 against 0.1177). It fits the training noise harder. Choosing the learning rates on validation loss instead (8 fresh datasets) shrinks the deficit to +0.0038 (95% CI [+0.0032, +0.0044]) against SGD + momentum, cosine lr, SVRG with plain momentum ties the best baseline (+0.0000, CI [-0.0001, +0.0007]), and Adam is itself worse than SGD by +0.0012: the deficit tracks the adaptive scaling, not the removal of gradient variance. A sharp-versus-flat-minima explanation was not tested. For held-out logistic regression all methods are indistinguishable (test log-loss 0.68775 to 0.68776, about 54% accuracy).

n	d	normal equations (s)	`lstsq` (s)	Jacobi SVRG (s)	Adam, untuned
100 000	256	0.163	0.173	1.911	not reached
200 000	1024	0.153	0.724	0.943	not reached
200 000	2048	0.490	1.758	0.488	not reached

Table 5: GPU wall-clock (Colab T4, float32, Gaussian features, target excess loss 10^{-4} of the initial excess). Jacobi SVRG is the PyTorch engine with a frozen preconditioner, telemetry off, fixed untuned hyper-parameters.

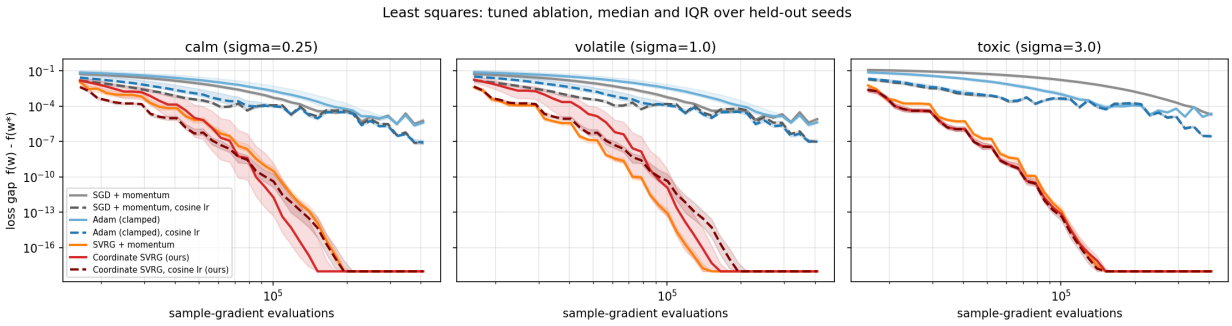


Figure 1: Least squares, three noise regimes: loss gap against sample-gradient evaluations (median and IQR over held-out seeds). Curves are clamped at 10^{-18} for plotting.

5.5 Snapshot interval and wall-clock

Evaluations to the target are U-shaped in the snapshot interval K: best at $K = 64$ (60,800 evaluations), within a factor 2.1 for $16 \le K \le 256$, 2.2× worse at $K = 8$ and 5.6× worse at $K = 1024$ (Figure 4). On a larger problem ($n = 40000$, $d = 200$) SVRG with momentum reaches the target in 0.08 s on one NumPy core, whereas the normal equations take 0.06 s and `lstsq` 0.48 s: a direct solve still matches SVRG at this size, and the iterative timing excludes the cost of tuning the learning rate.

5.6 A method read off the theory

Proposition 5 suggests using the frozen Jacobi preconditioner $D = \mathrm{diag}(X^\top X/n)$ and taking every other hyper-parameter from the Johnson–Zhang recipe: $\eta = 0.1/L_D$, $m = \lceil 50 L_D/\gamma_D \rceil$ single-sample inner steps ($m = 4{,}139$ here), random-iterate snapshots, for a guaranteed per-epoch contraction factor $\alpha = 0.50$ and *no tuning*. On the datasets and seeds of Tables 1 and 2 (Table 3), this reaches 2.6×10^{-29} on the badly scaled problem, at float64 round-off, beating Coordinate SVRG with cosine decay (3.5×10^{-11}) on 20/20 seeds and needing 73,848 sample-gradient evaluations to reach the target against 348,288 for that method (4.7× fewer). The measured mean epoch-to-epoch contraction is 0.08 (badly scaled) and 0.10 (well-scaled), consistent with the guaranteed 0.50 in expectation but loose by a factor of about five. On well-scaled data it matches the other round-off-level methods. Does the finding carry beyond least squares? On ℓ_2-regularised logistic regression with the same three-decade feature scaling (12 held-out seeds, Hessian-diagonal upper bound $D = \frac{1}{4}\,\mathrm{mean}(x^2) + \ell_2$, learning rate tuned like every other method), tuned Jacobi SVRG again reaches round-off (median 9.7×10^{-30}) and beats Coordinate SVRG with cosine decay (7.4×10^{-19}) on 12/12 seeds, needing 169,152 evaluations to the target against 485,296 (2.9× fewer); tuned Adam with cosine decay stops at 8.7×10^{-7}. Two nuances: this run is tuned, not theory-prescribed,

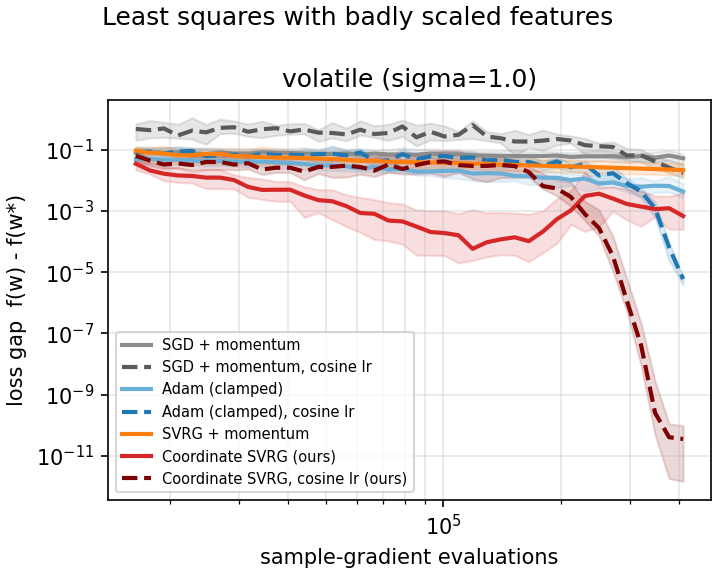


Figure 2: Badly scaled features. Among these methods, only variance reduction with adaptive scaling and a decaying rate reaches the target; Section 5.6 adds a simpler method that does better.

because the global strong-convexity modulus of the logistic loss is only the ℓ_2 weight and the worst-case recipe is far too conservative to be informative; and on this problem constant-rate Coordinate SVRG does *not* plateau and has a paired difference of -9.3 decades against SVRG with plain momentum, so the plateau seen in the least-squares case is problem-dependent. The comparison favours Jacobi-SVRG in one respect that should be stated: it uses the Hessian diagonal of a linear model, one extra pass over the data not charged above, whereas Coordinate SVRG uses gradients only and applies to models where the Hessian diagonal is unavailable. The finding is that where the Hessian diagonal *is* available, the simpler theory-derived method dominates the more elaborate one.

5.7 Real data with raw features

To check that the findings are not artefacts of the simulator, I ran the same protocol on three bundled real datasets with raw, unstandardised features plus an intercept (Table 4): breast-cancer diagnosis and wine (logistic regression) and diabetes progression (least squares). Their feature scales span 5.3 decades (breast cancer) down to 1.8 decades, and Jacobi scaling reduces the condition number of the Hessian at the optimum by two to three orders of magnitude (breast cancer: $3.1 \times 10^7 \to 2.5 \times 10^5$). Two results carry over and one does not. Preconditioned methods (Jacobi SVRG, Coordinate SVRG, Adam) beat the unpreconditioned ones (SGD, SVRG with momentum) by large margins on all three. Jacobi SVRG has the lowest median gap on wine and diabetes and is tied with Coordinate SVRG with cosine decay on breast cancer (medians differing by 0.2%); its margin over the best tuned baseline is small, between 0.06 and 0.52 decades. What does *not* carry over, with raw features, is the round-off result: no method reached the 10^{-8} target within 1000 epochs on any dataset. Experiment 16 shows why. Standardising the same features to unit variance cuts the Hessian condition numbers to 140, 34 and 470, and then SVRG variants reach the target on 3/3 datasets (the best of them reaching at worst 2.2×10^{-20}), while the best tuned baselines stop between 6.1×10^{-8} and 1.8×10^{-2} and reach it on 0/3. The synthetic-data conclusion therefore

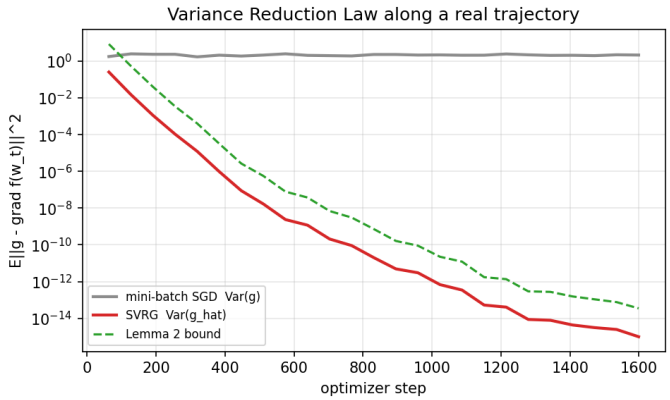


Figure 3: Gradient variance along the trajectory, with the Lemma 2 bound.

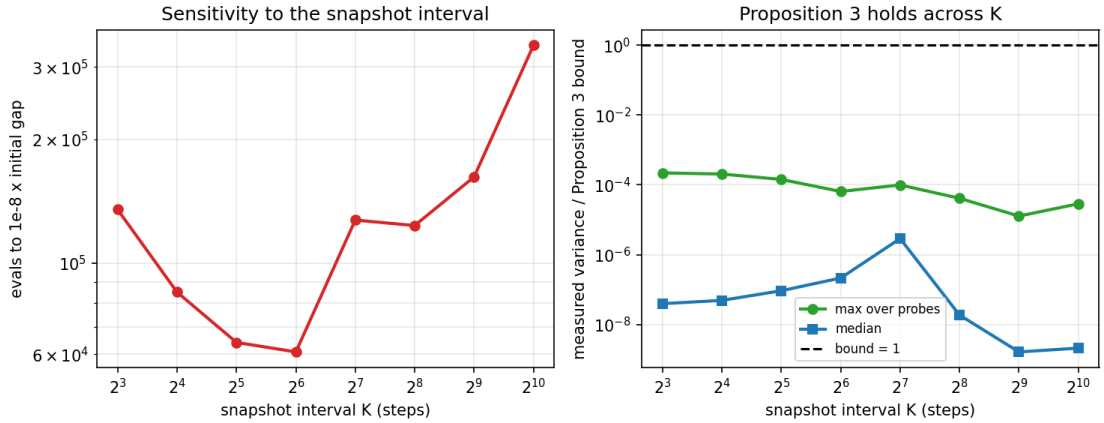


Figure 4: Sensitivity to the snapshot interval, and the Proposition 3 ratio across intervals.

reproduces on real data once scale ill-conditioning is removed; on standardised features the Jacobi preconditioner is close to the identity and offers no advantage over SVRG with momentum. Three datasets cannot support a general claim, the seeds vary only the batch order (so the intervals say nothing about variation across datasets), and one baseline learning rate (diabetes, Adam with cosine decay) sat on the edge of its grid.

5.8 Walk-forward signal tracking: no benefit

In a rolling re-estimation of a drifting alpha under a compute budget, every method's out-of-sample correlation with the clean signal differs from closed-form OLS by at most 0.0054, against a spread across datasets of about 0.08; paired intervals often exclude zero but the effects are negligible, and SVRG never beats Adam. The problem is too small for the choice of optimizer to matter.

5.9 Per-operation cost in compiled code

An external review cited the 2 ms GPU step (Colab T4, telemetry on, 32 parameters) as evidence that the method is too slow for latency-sensitive use. That figure measures PyTorch per-step overhead, not the algorithm (Section 5.10 shows that overhead is about 1.6 ms per step even with

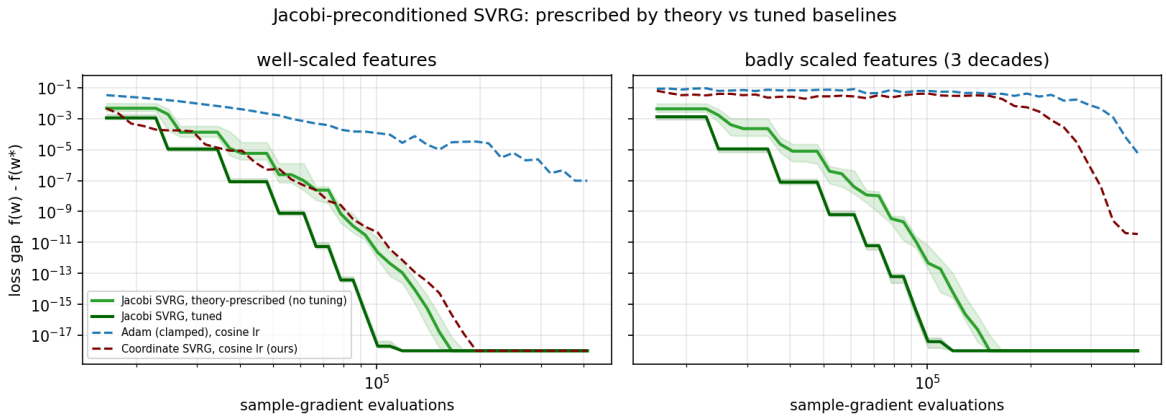


Figure 5: Jacobi-preconditioned SVRG (theory-prescribed and tuned) against the best tuned baselines.

telemetry off). A compiled C implementation of the same update (frozen Jacobi preconditioner), which reproduces the NumPy oracle to 3.3×10^{-16}, costs 129 ns per single-sample step at $d = 32$ and 3.9 μs for a 64-sample batch on one core of an Intel Xeon @ 2.10GHz cloud CPU, against 959 ns for a recursive-least-squares update and 15 ns for the signal-inference dot product (on a Colab CPU: 217 ns, 1.5 μs and 20 ns); one snapshot pass over a 100,000-row window costs 1.8 ms. At $d = 128$ a single-sample SVRG step is 43× cheaper than an RLS update ($O(bd)$ versus $O(d^2)$). This shows the algorithm is not intrinsically slow. It does not make this repository a production system (no thread pinning, lock-free queues or fixed-point arithmetic), RLS is exact at every tick so the speed comparison is not a claim of advantage, and fitting and inference are different paths: latency budgets of tens of microseconds apply to inference, while a refit on a trailing window can run off the critical path. The numbers are machine-specific and measured on shared hardware.

5.10 GPU wall-clock against direct solves

Table 5 reports a Colab T4 run (float32, Gaussian features, which are well conditioned and so the best case for SVRG; fixed, untuned hyper-parameters; telemetry off; convergence checks outside the timed region). Jacobi SVRG, the PyTorch engine with a frozen preconditioner, reached the target in 3 epochs at every size. Direct normal equations were faster at $d = 256$ (0.163 s against 1.91 s) and $d = 1024$ (0.153 s against 0.94 s), and the two tied at $d = 2048$ (0.490 s against 0.488 s); `lstsq` was 3.6× slower than SVRG there and used up to 3.3 GB of extra memory against 9 MB at most for SVRG. SVRG's time fell as d grew because the batch size ($\max(256, d)$) grew while the per-step overhead stayed near 1.6 ms, so on this hardware the crossover with the best direct method is at about $d = 2048$ for $n = 200{,}000$; larger sizes were not tested. Untuned Adam did not reach the target within 60 epochs at any size (reached: no), so it is a weak baseline here. The engine's per-step cost does not depend on problem size from $d = 256$ to 2048: 1.6 ms with telemetry off, 1.9 ms lazy and 2.0 ms eager (eager costs about 1.2× the telemetry-off step; lazy saves about 6%). Host synchronisation from telemetry is therefore *not* the dominant cost, contrary to what I had assumed earlier: the floor is per-step framework overhead (kernel launches and autograd for the two gradient evaluations).

5.11 Real market data: a walk-forward study of ten ETFs

I ran the walk-forward study on 10 liquid ETFs (daily, Yahoo Finance, 2008–2025; 4,528 days, 3,749 out-of-sample days) with nine causal features (lagged returns, realised volatility, a volume z-score), a 756-day window refitted every 21 days, a dollar-neutral long-short strategy and 5 bps cost per unit traded; the full report, with a placebo run on circularly shifted targets, is in `results/real_market/report.md`. **There is no detectable tradable signal.** The mean daily cross-sectional rank IC of the fitted models lies between -0.0022 and $+0.0079$, inside the range of the placebo (-0.0091 to $+0.0090$). The pooled IC is weakly positive ($+0.0187$ for OLS and $+0.0292$ for ridge, against at most 0.0043 in absolute value in the single placebo draw), but the net-of-cost Sharpe ratio is negative for every fitted model (best -0.76; OLS -1.21 at a turnover of 1.09 per day). **The optimizer does not matter.** Budget-limited Jacobi SVRG reproduces OLS to four decimals (IC difference $+0.0000$, 95% CI $[-0.0000, +0.0001]$) and budget-limited Adam is statistically indistinguishable from it ($+0.0004$, CI $[-0.0078, +0.0087]$): on this small, well-conditioned problem the iterative methods converge to the closed-form solution. **Regularisation does:** validated ridge beats OLS by $+0.0101$ in mean daily IC (CI $[+0.0012, +0.0188]$) and by $+0.45$ in net Sharpe (CI $[+0.20, +0.69]$), though every net Sharpe remains negative. Caveats: ten ETFs, daily data, simple features and a flat cost assumption; one placebo draw, so the placebo range is itself uncertain; no multiple-comparison correction; no claim that other assets, frequencies or features would behave the same. This is the only market data analysed, and it supports no claim of a tradable edge.

6 Ablating the invariants

Sample alignment (invariant 2). Taking the snapshot gradient on an *independent* batch instead of the same one destroys the control variate (Figure 6). At snapshot distance 0.01 the estimator's mean squared error is 5.1×10^{-5} with aligned batches, 3.4 with misaligned batches, and 1.6 for plain mini-batch SGD: misaligned SVRG is $2.1\times$ *worse than not using variance reduction*, and its error does not shrink with distance.

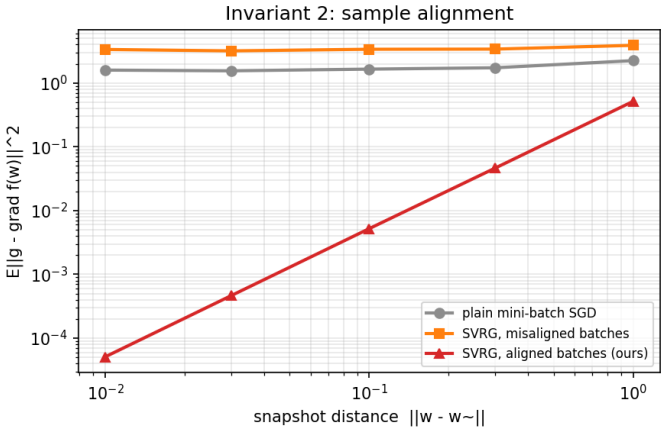


Figure 6: Gradient-estimator error against snapshot distance, with aligned and misaligned batches.

Coordinate safeguards (invariant 4). In 2,000 adversarial float32 trials (10 steps, momentum and decay up to 0.9999, gradients drawn from zeros, denormals and values up to 3×10^{38}, and in a

second tier also infinities and NaN), textbook Adam with an additive ε produced non-finite weights in 76% of trials with extreme but finite inputs and 100% with infinities or NaN; a clamp-only variant failed identically (76%); the full guard set failed in 0%. The operative guard is therefore the sanitising and bounding of the gradient; the denominator floor alone adds nothing in this test. (An earlier version of this work claimed that clamping rather than offsetting the denominator mattered; this experiment shows it does not.) The inputs are deliberately hostile and describe a robustness guarantee, not typical training.

7 Limitations and threats to validity

- **Mostly synthetic data.** The simulator generates what the models assume, the only real data are three small non-financial datasets (on which the round-off results reproduce only after the features are standardised) and one walk-forward study of ten ETFs, which found no tradable signal. Nothing here is evidence of a tradable edge.
- **Convex bias.** Most experiments are convex, the best case for SVRG; variance reduction is known to help far less in deep learning [4]. One small non-convex network is not evidence about deep models.
- **Scale.** The largest problem has $n = 40000$, $d = 200$, where a direct solve is as fast as any iterative method.
- **Engines versus experiments.** The PyTorch and JAX engines passed the full 68-test suite, including step-for-step agreement with the oracle, on a Colab T4 runtime; on the GPU the PyTorch engine stayed within 1.8×10^{-7} of the oracle over 100 steps, and a 640-step benchmark's final loss gaps matched the oracle to four significant digits. The reported experiment numbers nevertheless come from the NumPy oracle; timing is reported separately in Sections 5.9 and 5.10.
- **No convergence proof for the adaptive engine**; the cosine-decay results are empirical.
- **Statistics.** Seed counts range from 3 to 20; intervals are bootstrap intervals over seeds; no multiple-comparison correction; learning rates are tuned on training loss in the non-convex and logistic held-out checks.
- **Scope.** This is an offline research and calibration study, not a production low-latency component; the compiled-kernel timings (Section 5.9) measure the algorithm on one CPU core, not a deployable system.
- **Tuning cost** is excluded from the iterative wall-clock times.
- **Jacobi-SVRG scope.** It needs the Hessian diagonal, available for linear and generalised-linear models but not in general. It was tested on least squares (theory-prescribed and tuned) and logistic regression (tuned only), its single-sample theory variant is slow per evaluation in wall-clock terms, and the evaluation counts above do not charge the one extra pass that computes D.

8 Reproducibility

The repository fixes every random stream from one seed. `make test` runs the invariant, oracle, differential and fuzz tests; `make experiments` regenerates all results and figures; `make report` rebuilds this document, whose numbers are macros generated from `results/experiments.json`.

A continuous-integration workflow runs the tests on CPU PyTorch and JAX. A purged walk-forward study on real daily returns, with a placebo run, is provided as a script in the `experiments` folder (`realmarketstudy`); its results are reported in Section 5.11.

References

[1] R. Johnson, T. Zhang. Accelerating stochastic gradient descent using predictive variance reduction. *NeurIPS*, 2013.

[2] D. Kingma, J. Ba. Adam: a method for stochastic optimization. *ICLR*, 2015.

[3] S. Reddi, S. Kale, S. Kumar. On the convergence of Adam and beyond. *ICLR*, 2018.

[4] A. Defazio, L. Bottou. On the ineffectiveness of variance reduced optimization for deep learning. *NeurIPS*, 2019.

[5] A. van der Sluis. Condition numbers and equilibration of matrices. *Numerische Mathematik* 14, 1969.